



Lesson 1: From Blocks to Words — Hello, Python!

Big Idea

Everything you built in Scratch with colorful blocks, a computer can also do with typed words — and today we learn to speak that language: Python.

Learning Objectives

By the end of this lesson, students will be able to:

- Explain that Python is a way of giving the computer instructions using typed text instead of blocks.
- Open and use Thonny (or Repl.it) to write and run a Python program.
- Write and run a `print()` statement to display a message.
- Compare a simple Scratch script to the equivalent Python code.
- Fix a basic syntax mistake (like a missing quote or missing parenthesis).

Teacher Preparation

- Install/open Thonny on all student machines, OR have Repl.it accounts ready as a backup.
- Have a simple Scratch project ready to show on screen (a sprite that says "Hello!").
- Print or display the "Scratch vs Python" comparison poster (optional, can be drawn on whiteboard).
- Test that Thonny's "Run" button (green play arrow) works on the lab machines beforehand.

Required Files

- `Student_Lesson_Guide.md`
- `Activities.md`
- `Challenges.md`
- Thonny IDE or a Repl.it project set to Python 3

Teaching Flow (approx. 45–50 minutes)

1. Warm-up / Hook — 5 min
2. Concept Introduction — 8 min
3. Teacher Demonstration — 7 min

4. Guided Coding — 10 min
5. Mini Challenge — 5 min
6. Main Activity — 10 min
7. Check for Understanding + Recap — 5 min

Warm-up / Hook (5 min)

Show the class a Scratch project with a sprite that says "Hello, world!" when the green flag is clicked. Ask: "What block made the sprite talk?" (Answer: the "say" block.) Then say: "Today, we're going to make words appear on the screen too — but instead of dragging a block, we're going to TYPE a magic word: `print`."



Coding Mission: Today's mission is to make the computer say hello — in text, not blocks!

Concept Introduction

Explain gently, using simple analogies: - "In Scratch, you snapped together puzzle pieces to tell the sprite what to do. In Python, we type those same instructions as words and symbols — it's just a different way of talking to the computer." - "Python is a programming *language*, just like English or Spanish is a language for people. Python is a language for talking to computers." - "The `print()` command is Python's version of the Scratch 'say' block. Whatever you put inside the parentheses gets shown on the screen." -  **Robot Tip:** "Computers are very literal — they do exactly what you type, so we have to be exact with our spelling, quotes, and parentheses!"

Teacher Demonstration (live-type these, step by step)

1. Open Thonny. Show the two panels: the **editor** (top, where we write code) and the **shell** (bottom, where we see output).
2. Type this line in the editor: `python print("Hello, world!")`
3. Click the green **Run** button (or press F5).
4. Point out that "Hello, world!" appeared in the shell at the bottom.
5. Now break it on purpose — delete the closing quote — and run again, so students see a red error message. Say: "See? Python tells us when something doesn't look right!"
6. Fix the quote back and run again to show it works.

Guided Coding (students type along)

Have students type this exact program into their own Thonny editor:

```
print("Hello, world!")  
print("My name is Python, and I love to help you build things!")
```

Walk around and check that everyone's quotes and parentheses match up before they press Run.

Mini Challenge 🎮

Ask students to change the message inside the first `print()` so it says hello using their own name, for example:

```
print("Hello, my name is Maya!")
```

Then run it and check the shell shows their own greeting.

Main Activity / Project

Students complete the "My First Python Program" activity in `Activities.md`, writing three `print()` lines that introduce themselves (name, favorite color, favorite animal).

Fun Challenge (optional, harder) ★

Challenge advanced students to print a small ASCII "picture" using multiple `print()` lines, such as a simple rocket made of dashes and letters, e.g.:

```
print("  /\  ")  
print(" /  \ ")  
print(" |    | ")  
print(" |PYTHON| ")  
print("/_____\")
```

Check for Understanding (ask out loud)

- "What Scratch block is like Python's `print()`?"
- "What two symbols do we always need around our message?" (quotes)
- "What happens if we forget the closing parenthesis?"
- "Where do we click to run our code in Thonny?"

Common Student Mistakes

- **Forgetting quotes:** `print(Hello!)` → Python doesn't know what `Hello!` is. Fix: wrap text in quotes: `print("Hello!")`.
- **Mismatched quotes:** starting with `"` and ending with `'`. Fix: always match the same quote type on both sides.
- **Missing parenthesis:** `print("Hi!")` → Python waits for more. Fix: always close with `)`.
- **Typos in the word `print`:** e.g. `Print` (capital P) causes an error in most contexts. Fix: Python is case-sensitive — always type `print` in lowercase.
- **Forgetting to click Run:** students type code but never see output. Fix: remind them to press the green Run button or F5 every time.

Differentiation

- **Struggling support:** Provide a pre-typed line for them to copy exactly first, then only change the name inside the quotes.
- **Standard:** Complete the Guided Coding and Main Activity as written.
- **Extension for advanced learners:** Try the Fun Challenge ASCII art, or research (with teacher help) what `print()` does with numbers instead of text (e.g. `print(5 + 3)`).

Recap

"Today we learned that Python is just another way to give the computer instructions — the same kind of instructions we used to give with Scratch blocks! We used `print()` to make the computer show a message, just like the 'say' block did in Scratch."

Take-Home / Creative Challenge

Ask students to write (on paper or on a home computer if available) three `print()` lines that would introduce their favorite book, movie, or game character, and bring it to share next class.

Teacher Talking Points

- "Remember how your Scratch sprite said 'Hello' when you clicked the green flag? Well, guess what — we can do that exact same thing by typing just one line of code!"
- "Python doesn't care about being fancy, but it DOES care about spelling and punctuation being exact — kind of like a very picky friend who needs everything just right."

- "Don't worry if you see a red error message. That's not scary — it's Python's way of raising its hand and saying 'hey, I need a little help understanding you!'"
- "You are not just typing words — you are now officially coders who write real code, the same kind used to build apps, games, and websites!"



Victory Moment

Celebrate the first successful run of `print("Hello, world!")` as a big class win — this is the same first line every programmer in the world has typed at some point!